ACM 竞赛常用模板手册

基础头文件 (Boilerplate)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using vll = vector<ll>;
using vint = vector<int>;

void _(){}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr), cout.tie(nullptr);
    int t; cin >> t;
    while (t--) _();
}
```

> `sync_with_stdio(false)` 关闭 C 风格 IO 同步，`cin.tie(0)` 解绑 cin 和 cout，大幅提升输入输出速度。

1. 快速幂 (Binary Exponentiation)

> 在 O(log b) 时间内计算 `a^b mod p`，广泛用于需要模意义下幂运算的场景。

```
ll qpow(ll a, ll b, ll p) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % p;
        a = a * a % p , b >>= 1;
    }
    return res;
}
```

2. 费马小定理求逆元 (Modular Inverse)

> **定理：** 若 p 是质数且 a 不被 p 整除，则 a^(p-1) ≡ 1 (mod p)。
>
> 由 a · a^(p-2) ≡ 1 (mod p) 可知，**a 的逆元 = a^(p-2) mod p**，即用快速幂在 O(log p) 内求出。
>
> **常用质数模数：** `1e9+7`, `998244353`

```
ll inv(ll a, ll p) {return qpow(a, p - 2, p); } // p 必须是质数
```

---

3. 线性预处理阶乘 & 组合数

> 预处理好阶乘 fact[] 和逆阶乘 invfact[]，之后 O(1) 回答组合数 C(n, k)。

```
const int mod = 1e9 + 7;
const int N = 1e5 + 10;
ll fact[N], inv[N];

void prework(int n) {
    fact[0] = fact[1] = 1;
    for (int i = 2; i <= n; i++)
        fact[i] = fact[i - 1] * i % mod;
    inv[n] = qpow(fact[n], mod - 2, mod);
    for (int i = n - 1; i >= 0; i--)
        inv[i] = inv[i + 1] * (i + 1) % mod;
}

ll C(int n, int k) {
    if (k > n || k < 0) return 0;
    return fact[n] * inv[k] % mod * inv[n - k] % mod;
}
```

---

4. 最大公约数 & 扩展欧几里得 (GCD & exGCD)

```
ll gcd(ll a, ll b) { return b ? gcd(b, a % b) : a; }

//recommend
ll gcd(ll a, ll b) {int t;while(t = a % b) a = b , b = t; return b}

ll lcm(ll a, ll b) { return a / gcd(a, b) * b; }

// ax + by = gcd(a, b), 求一组 x, y
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) { x = 1; y = 0; return a; }
    ll g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}

// exgcd 求 a 的逆元 (模 p, p 可不质数, 但需 gcd(a,p)=1)
ll exinv(ll a, ll p) {
    ll x, y;
    exgcd(a, p, x, y);
```

```
    return (x % p + p) % p;
}
```

---

5. 欧拉筛 (线性筛素数)

> O(n) 筛出 1~n 内所有素数，并求出每个数的最小质因子。

```
const int N = 1e6 + 10;
int primes[N], cnt;
bool st[N];

void get_primes(int n) {
    for (int i = 2; i <= n; i++) {
        if (!st[i]) primes[cnt++] = i;
        for (int j = 0; primes[j] <= n / i; j++) {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}
```

---

6. 树状数组 (Fenwick Tree / BIT)

- fwt 好名字

> 支持 **单点加 + 前缀和查询**，O(log n)。注意树状数组下标从 1 开始。

```
struct BIT {
    ll t[N]; int n;

    void init(int n_) { n = n_; }

    int lowbit(int x) { return x & -x; }

    void add(int x, ll k) {
        for (; x <= n; x += lowbit(x)) t[x] += k;
    }

    ll sum(int x) {                 // 前缀和 [1, x]
        ll res = 0;
        for (; x; x -= lowbit(x)) res += t[x];
        return res;
    }

    ll range(int l, int r) {        // 区间和 [l, r]
        return sum(r) - sum(l - 1);
    }
```

```
};

// 区间加 + 单点查询：维护一个差分 BIT，对区间 [l,r]+k:
// add(l, k), add(r+1, -k)，单点值 = sum(x)
//
// 区间加 + 区间和查询：维护两个 BIT 分别存 t1[i]*i 和 t1[i]
```

---

7. 线段树 — 区间加、区间求和 (Segment Tree + Lazy)

> O(log n) 支持区间修改和区间查询。

```
const int N = 1e5 + 10;
ll a[N];  // 原始数组（下标从 1 开始）

struct SegTree {
    ll sum[N << 2], tag[N << 2];

    void pushup(int u) { sum[u] = sum[u << 1] + sum[u << 1 | 1]; }

    void apply(int u, int len, ll k) {
        sum[u] += k * len;
        tag[u] += k;
    }

    void pushdown(int u, int l, int r) {
        if (!tag[u]) return;
        int mid = (l + r) >> 1;
        apply(u << 1, mid - l + 1, tag[u]);
        apply(u << 1 | 1, r - mid, tag[u]);
        tag[u] = 0;
    }

    void build(int u, int l, int r) {
        //build from a[]

        if (l == r) { sum[u] = a[l]; return; }
        int mid = (l + r) >> 1;
        build(u << 1, l, mid);
        build(u << 1 | 1, mid + 1, r);
        pushup(u);
    }

    void update(int u, int l, int r, int ql, int qr, ll k) {
        if (ql <= l && r <= qr) { apply(u, r - l + 1, k); return; }
        pushdown(u, l, r);
        int mid = (l + r) >> 1;
        if (ql <= mid) update(u << 1, l, mid, ql, qr, k);
        if (qr > mid)  update(u << 1 | 1, mid + 1, r, ql, qr, k);
        pushup(u);
    }
```

```
    ll query(int u, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return sum[u];
        pushdown(u, l, r);
        int mid = (l + r) >> 1;
        ll res = 0;
        if (ql <= mid) res += query(u << 1, l, mid, ql, qr);
        if (qr > mid)  res += query(u << 1 | 1, mid + 1, r, ql, qr);
        return res;
    }
};
```

---

8. 并查集 (DSU / Union-Find)

> 近乎 O(1) 的合并与查找，维护连通性、判环等。

```
struct DSU {
    int fa[N], sz[N];

    void init(int n) {
        for (int i = 1; i <= n; i++) fa[i] = i, sz[i] = 1;
    }

    int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }

    void merge(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return;
        if (sz[ra] < sz[rb]) swap(ra, rb);  // 按秩合并
        fa[rb] = ra;
        sz[ra] += sz[rb];
    }

    bool same(int a, int b) { return find(a) == find(b); }
};
```

---

9. Trie (字典树)

> O(|S|) 插入/查询字符串前缀，用于字符串检索、最大异或对等。

```
struct Trie {
    int nex[N * 26][26], idx;
    int cnt[N * 26];  // 经过该节点的字符串数量

    void insert(string &s) {
        int p = 0;
        for (char c : s) {
```

```
            int u = c - 'a';
            if (!nex[p][u]) nex[p][u] = ++idx;
            p = nex[p][u];
            cnt[p]++;
        }
    }

    int query(string &s) {  // 查询 s 作为前缀出现的次数
        int p = 0;
        for (char c : s) {
            int u = c - 'a';
            if (!nex[p][u]) return 0;
            p = nex[p][u];
        }
        return cnt[p];
    }
};
```

10. 字符串单哈希 / 双哈希

> O(n) 预处理，O(1) 子串哈希比较，用于快速判字符串相等。

```
using ull = unsigned long long;
const ull B = 131;  // 大于字符集大小的质数

struct StringHash {
    ull h[N], p[N];  // p[i] = B^i

    void init(string &s) {
        int n = s.length();
        p[0] = 1;
        for (int i = 1; i <= n; i++) {
            h[i] = h[i - 1] * B + s[i - 1];
            p[i] = p[i - 1] * B;
        }
    }

    ull get(int l, int r) {  // 1-indexed, 闭区间 [l,r]
        return h[r] - h[l - 1] * p[r - l + 1];
    }
};

// 双哈希防冲突版本（双模数）
const ull B1 = 131, M1 = 1e9 + 7;
const ull B2 = 13331, M2 = 1e9 + 9;
// 使用 pair<ull, ull> 代替 ull，规则同上
```

11. KMP 字符串匹配

> O(n+m) 在文本串 T 中找模式串 P 的所有出现位置。

```
// nxt[i]: P[0..i-1] 的最长相等真前后缀长度
// 求 next 数组: O(m)
void get_next(string &P, vector<int> &nxt) {
    int m = P.length();
    nxt.resize(m + 1);
    nxt[0] = -1;
    for (int i = 0, j = -1; i < m; ) {
        if (j == -1 || P[i] == P[j]) nxt[++i] = ++j;
        else j = nxt[j];
    }
}

// 匹配: 返回所有匹配的起始位置 O(n+m)
vector<int> kmp(string &T, string &P) {
    vector<int> nxt, res;
    get_next(P, nxt);
    int n = T.length(), m = P.length();
    for (int i = 0, j = 0; i < n; ) {
        if (j == -1 || T[i] == P[j]) i++, j++;
        else j = nxt[j];
        if (j == m) {
            res.push_back(i - m);  // 0-indexed 起始位置
            j = nxt[j];
        }
    }
    return res;
}
```

---

12. Dijkstra 最短路

> 非负权图单源最短路，时间复杂度 O((n+m) log n)。

```
struct Edge { int v; ll w; };
vector<Edge> g[N];
ll dist[N];
bool vis[N];

void dijkstra(int s, int n) {
    using P = pair<ll, int>;  // {dist, node}
    priority_queue<P, vector<P>, greater<P>> pq;
    fill(dist + 1, dist + n + 1, 1e18);
    fill(vis + 1, vis + n + 1, false);

    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        int u = pq.top().second; pq.pop();
```

```
        if (vis[u]) continue;
        vis[u] = true;
        for (auto &e : g[u]) {
            if (dist[e.v] > dist[u] + e.w) {
                dist[e.v] = dist[u] + e.w;
                pq.push({dist[e.v], e.v});
            }
        }
    }
}
```

---

13. 拓扑排序 (Topological Sort)

> 对 DAG 拓扑排序，可用于判环（最后入队节点数 < n 则有环）。

```
int in[N];
vector<int> g[N];

vector<int> topo(int n) {
    vector<int> res;
    queue<int> q;
    for (int i = 1; i <= n; i++)
        if (!in[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        res.push_back(u);
        for (int v : g[u])
            if (--in[v] == 0) q.push(v);
    }
    return res;  // 若 res.size() < n 则图中有环
}
```

---

14. 分数规划 / 二分模板

```
// 实数二分 (60 次迭代精度可达 1e-12)
auto ok = [&](double mid) -> bool {
    // 判断答案是否可行
};

double l = 0, r = 1e9;
for (int i = 0; i < 60; i++) {
    double mid = (l + r) / 2;
    if (ok(mid)) l = mid;
    else r = mid;
}
// 最终答案: l 或 r
```

15. ST 表 (RMQ)

> O(n log n) 预处理，O(1) 查询区间最值（静态）。

```
int st[N][20];  // st[i][j]: 从 i 开始长为 2^j 的最值

void build(int n) {
    for (int j = 1; j < 20; j++)
        for (int i = 1; i + (1 << j) - 1 <= n; i++)
            st[i][j] = max(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
}

int query(int l, int r) {
    int k = __lg(r - l + 1);  // __lg 是 C++17 的 log2
    return max(st[l][k], st[r - (1 << k) + 1][k]);
}

// 输入时直接把 a[i] 赋值给 st[i][0] 即可
```

常用技巧速查

场景	数据结构/算法
单点改 + 前缀和	树状数组 (BIT)
区间改 + 区间和	线段树 (lazy)
静态区间最值	ST 表
连通性 / 判环	并查集 (DSU)
字符串匹配	KMP / 哈希
非负权最短路	Dijkstra
负权最短路	SPFA / Bellman-Ford
求逆元 (质数模)	费马小定理: `a^(p-2) mod p`
求逆元 (任意模)	exGCD
log(N) 求幂	快速幂 `qpow`
O(1) 组合数	预处理阶乘 + 逆阶乘
字符串前缀检索 / 异或对	Trie
DAG 上的顺序	拓扑排序
二分答案 / 分数规划	实数二分 50-60 轮